

Specification of Source §1—2021 edition

Martin Henz, Lee Ning Yuan, Daryl Tan

National University of Singapore
School of Computing

July 3, 2026

The language Source is the official language of the textbook *Structure and Interpretation of Computer Programs, JavaScript Adaptation*. Source is a sublanguage of ECMAScript 2018 (9th Edition) and defined in the documents titled “Source §*x*”, where *x* refers to the respective textbook chapter.

1 Syntax

A Source program is a *program*, defined using Backus-Naur Form¹ as follows:

¹We adopt Henry Ledgard’s BNF variant that he described in *A human engineered variant of BNF*, ACM SIGPLAN Notices, Volume 15 Issue 10, October 1980, Pages 57-62. In our grammars, we use **bold** font for keywords, *italics* for syntactic variables, ϵ for nothing, $x \mid y$ for *x* or *y*, $[x]$ for an optional *x*, $x \dots$ for zero or more repetitions of *x*, and (x) for clarifying the structure of BNF expressions.

<i>program</i> ::= <i>import-directive... statement...</i>	program
<i>import-directive</i> ::= import { <i>import-names</i> } from <i>string</i> ;	import directive
<i>import-names</i> ::= ϵ <i>import-name</i> (, <i>import-name</i>)...	import name list
<i>import-name</i> ::= <i>name</i> <i>name as name</i>	import name
<i>statement</i> ::= const <i>name</i> = <i>expression</i> ;	constant declaration
function <i>name</i> (<i>names</i>) <i>block</i>	function declaration
return <i>expression</i> ;	return statement
<i>if-statement</i>	conditional statement
<i>block</i>	block statement
<i>expression</i> ;	expression statement
debugger ;	breakpoint
<i>names</i> ::= ϵ <i>name</i> (, <i>name</i>)...	name list
<i>if-statement</i> ::= if (<i>expression</i>) <i>block</i>	
else (<i>block</i> <i>if-statement</i>)	conditional statement
<i>block</i> ::= { <i>statement...</i> }	block statement
<i>expression</i> ::= <i>number</i>	primitive number expression
true false	primitive boolean expression
<i>string</i>	primitive string expression
<i>name</i>	name expression
<i>expression</i> <i>binary-operator</i> <i>expression</i>	binary operator combination
<i>unary-operator</i> <i>expression</i>	unary operator combination
<i>expression</i> <i>binary-logical</i> <i>expression</i>	logical composition
<i>expression</i> (<i>expressions</i>)	function application
(<i>name</i> (<i>names</i>)) => <i>expression</i>	lambda expression (expr. body)
(<i>name</i> (<i>names</i>)) => <i>block</i>	lambda expression (block body)
<i>expression</i> ? <i>expression</i> : <i>expression</i>	conditional expression
(<i>expression</i>)	parenthesised expression
<i>binary-operator</i> ::= + - * / % === !==	
> < >= <=	binary operator
<i>unary-operator</i> ::= ! -	unary operator
<i>binary-logical</i> ::= &&	logical composition symbol
<i>expressions</i> ::= ϵ <i>expression</i> (, <i>expression</i>)...	argument expressions

Restrictions

- Return statements are only allowed in bodies of functions.
- There cannot be any newline character between **return** and *expression* in return statements.²
- There cannot be any newline character between (*name* | (*parameters*)) and **=>** in function definition expressions.³
- Implementations of Source are allowed to treat function declaration as **syntactic sugar for constant declaration**.⁴ Source programmers need to make sure that functions are not called before their corresponding function declaration is evaluated.

Import directives

Import directives allow programs to import values from modules and bind them to names, whose scope is the entire program in which the import directive occurs. Import directives can only appear at the top-level. All names that appear in import directives must be distinct, and must also be distinct from all top-level variables. The Source specifications do not specify how modules are programmed.

Logical Composition

Conjunction

$$expression_1 \ \&\& \ expression_2$$

stands for

$$expression_1 \ ? \ expression_2 \ : \ \mathbf{false}$$

Disjunction

$$expression_1 \ || \ expression_2$$

stands for

$$expression_1 \ ? \ \mathbf{true} \ : \ expression_2$$

Names

Names⁵ start with `_`, `$` or a letter⁶ and contain only `_`, `$`, letters or digits⁷. Restricted words⁸ are not allowed as names.

Valid names are `x`, `_45`, `$$` and `π`, but always keep in mind that programming is communicating and that the familiarity of the audience with the characters used in names is an important aspect of program readability.

Numbers

We use decimal notation for numbers, with an optional decimal dot. “Scientific notation” (multiplying the number with 10^x) is indicated with the letter `e`, followed by the exponent *x*. Examples for numbers are `5432`, `-5432.109`, and `-43.21e-45`.

²Source inherits this syntactic quirk of JavaScript.

³ditto

⁴ECMAScript prescribes “hoisting” of function declarations to the beginning of the surrounding block. Programs that rely on this feature will run fine in JavaScript but might encounter a runtime error “Cannot access name before initialization” in a Source implementation.

⁵In [ECMAScript 2020 \(9th Edition\)](#), these names are called *identifiers*.

⁶By *letter* we mean [Unicode](#) letters (L) or letter numbers (NI).

⁷By *digit* we mean characters in the [Unicode](#) categories Nd (including the decimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9), Mn, Mc and Pc.

⁸By *restricted word* we mean any of: `arguments`, `await`, `break`, `case`, `catch`, `class`, `const`, `continue`, `debugger`, `default`, `delete`, `do`, `else`, `enum`, `eval`, `export`, `extends`, `false`, `finally`, `for`, `function`, `if`, `implements`, `import`, `in`, `instanceof`, `interface`, `let`, `new`, `null`, `package`, `private`, `protected`, `public`, `return`, `static`, `super`, `switch`, `this`, `throw`, `true`, `try`, `typeof`, `var`, `void`, `while`, `with`, `yield`. These are all words that cannot be used without restrictions as names in the strict mode of ECMAScript 2020.

Strings

Strings are of the form "*double-quote-characters*", where *double-quote-characters* is a possibly empty sequence of characters without the character " and without the newline character, of the form '*single-quote-characters*', where *single-quote-characters* is a possibly empty sequence of characters without the character ' and without the newline character, and of the form '*backquote-characters*', where *backquote-characters* is a possibly empty sequence of characters without the character '. Note that newline characters are allowed as *backquote-characters*.

The following characters can be represented in strings as given:

- horizontal tab: \t
- vertical tab: \v
- nul char: \0
- backspace: \b
- form feed: \f
- newline: \n
- carriage return: \r
- single quote: \'
- double quote: \"
- backslash: \\

Unicode characters can be used in strings using \u followed by the hexadecimal representation of the unicode character, for example '\uD83D\uDC04'.

Comments

In Source, any sequence of characters between “/*” and the next “*/” is ignored. After “//” any characters until the next newline character is ignored.

2 Dynamic Type Checking

Expressions evaluate to numbers, boolean values, strings or function values. Implementations of Source generate error messages when unexpected values are used as follows.

Only function values can be applied using the syntax:

$$expression ::= name(expressions)$$

For compound functions, implementations need to check that the number of *expressions* matches the number of parameters.

The following table specifies what arguments Source's operators take and what results they return. Implementations need to check the types of arguments and generate an error message when the types do not match.

operator	argument 1	argument 2	result
+	number	number	number
+	string	string	string
-	number	number	number
*	number	number	number
/	number	number	number
%	number	number	number
===	number	number	bool
===	string	string	bool
!==	number	number	bool
!==	string	string	bool
>	number	number	bool
>	string	string	bool
<	number	number	bool
<	string	string	bool
>=	number	number	bool
>=	string	string	bool
<=	number	number	bool
<=	string	string	bool
&&	bool	any	any
	bool	any	any
!	bool		bool
-	number		number

Preceding `?` and following `if`, Source only allows boolean expressions.

3 Standard Library

The standard library contains constants and functions that are always available in this language. The functions indicated as *primitive* are built into the language implementations. All others are considered *predeclared* and implemented using the primitive functions.

The standard library for Source § 1 is documented in online documentation of Source § 1.

Deviations from JavaScript

We intend the Source language to be a conservative extension of JavaScript: Every correct Source program should behave *exactly* the same using a Source implementation, as it does using a JavaScript implementation. We assume, of course, that suitable libraries are used by the JavaScript implementation, to account for the predefined names of each Source language.

This section lists some exceptions where we think a Source implementation should be allowed to deviate from the JavaScript specification, for the sake of internal consistency and esthetics.

Evaluation result of programs: JavaScript statically distinguishes between *value-producing* and *non-value-producing statements*. All declarations are non-value-producing, and all expression statements, conditional statements and assignments are value-producing. A block is value-producing if its body statement is value-producing, and then its value is the value of its body statement. A sequence is value-producing if any of its component statements is value-producing, and then its value is the value of its *last* value-producing component statement. The value of an expression statement is the value of the expression. The value of a conditional statement is the value of the branch that gets executed, or the value `undefined` if that branch is not value-producing. The value of an assignment is the value of the expression to the right of its `=` sign. Finally, if the whole program is not value-producing, its value is the value `undefined`.

Example 1:

```
1;
{
  // empty block
}
```

The result of evaluating this program in JavaScript is 1.

Example 2:

```
1;  
{  
  if (true) {} else {}  
}
```

The result of evaluating this program in JavaScript is `undefined`.

Implementations of Source are currently allowed to opt for a simpler scheme.

Hoisting of function declarations: In JavaScript, function declarations are “hoisted” (automagically moved) to the beginning of the block in which they appear. This means that applications of functions that are declared with function declaration statements never fail because the name is not yet assigned to their function value. The specification of Source does not include this hoisting; in Source, function declaration can be seen as syntactic sugar for constant declaration and lambda expression. As a consequence, application of functions declared with function declaration may fail in Source if the name that appears as function expression is not yet assigned to the function value it is supposed to refer to.